

Cryptography - 21_10

$$OWF \implies PRG \implies PRF$$

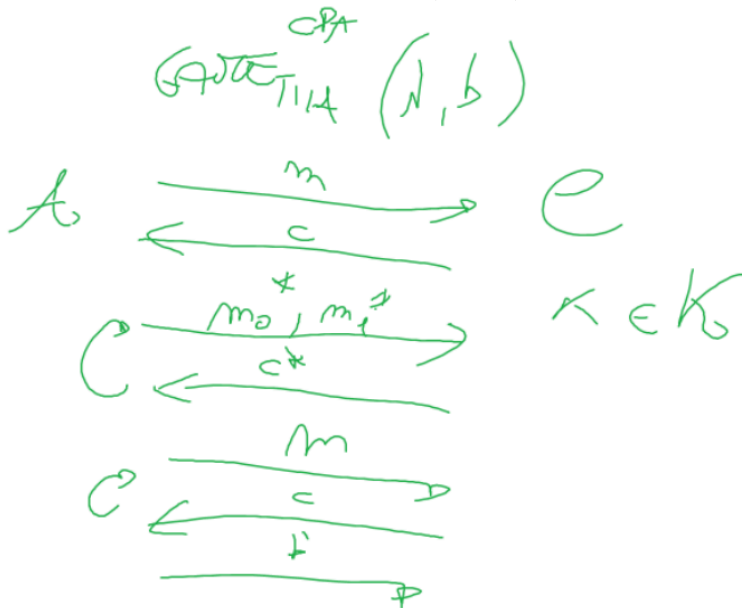
In the next few lectures, we'll see that PRFs are enough to do practical symmetric crypto:

- CPA-secure SKE for msg of variable input length
- Secure MACs for msg of variable input length (VIL).
- Non-malleable SKE (also known as CCA-secure SKE), which is basically equivalent to combining encryption and message authentication

As we are going to see, for some applications, it will be important that F is a PRP (Pseudorandom Permutation), namely it's invertible given the key. In practice, we call it a block cipher. We will show: $PRFs \implies PRPs$.

This will also explain real-world design of some block ciphers (DES, AES).

Let's start with encryption (CPA).

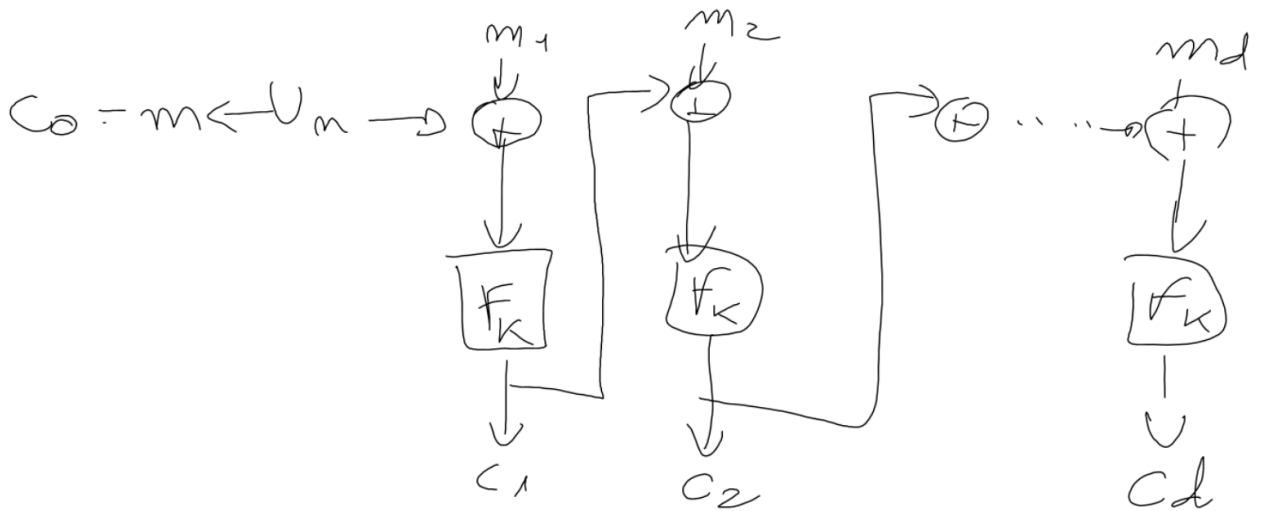


VIL: $|m|$ is any $poly(\lambda)$ such that $|m_0^*| = |m_1^*|$.

We will need a mode of operation: a standardized way to encrypt a msg $m = (m_1, \dots, m_d)$ where $d \in \mathbb{N}$ and $m_i \in \{0, 1\}^n$ (for example, $n = 256$) using a PRF $F = \{F_k : \{0, 1\}^n \rightarrow \{0, 1\}^n\}$.

Remark: We can't just output $C = (F_k(m_1), \dots, F_k(m_d))$ even assuming F is a PRP. This is known as ECB, and because it's deterministic we cannot use it.

CBC:



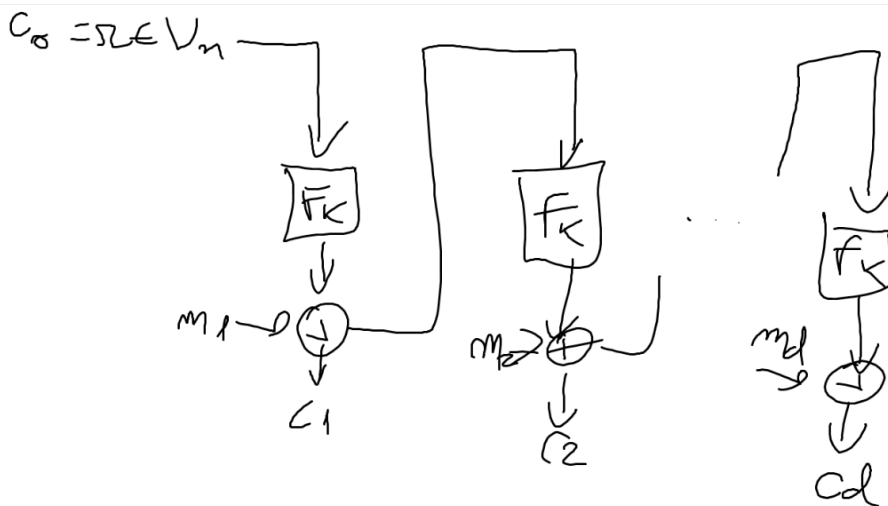
Thus getting $c = (c_0, c_1, \dots, c_d)$.

We have that $c_i = F_k(c_{i-1} \oplus m_i)$.

Some notes:

- Requires to invert F_k (so need a PRP).
- It's sequential
- It's secure: If F is a PRP, then CBC-mode is CPA-secure for VIL.

CFB:



Another variant is known as OFB, where, instead of chaining the output of $\oplus$, we feedback the output of F_k .

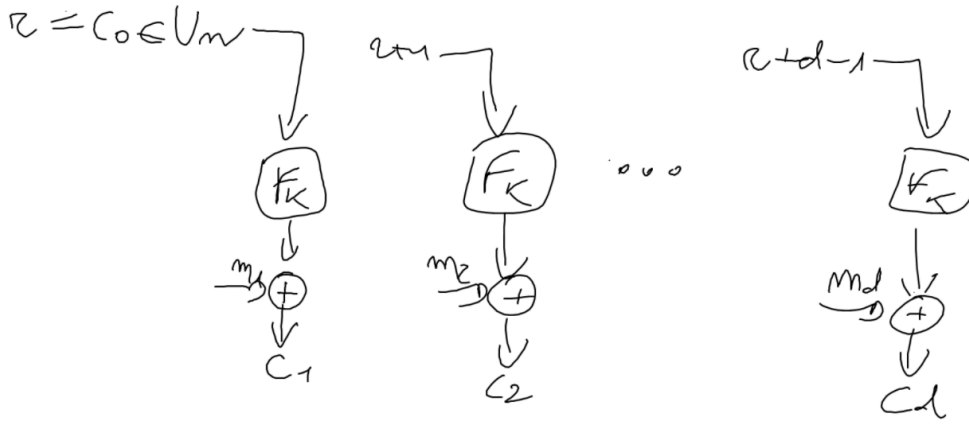
In CFB, $c_i = F_k(c_{i-1} \oplus m_i)$.

CTR:

Let "+" be addition (mod 2^n).

Here we start to count, so the value r originally chosen will be increased by 1 for

each message, however the addition is module 2^n .



✓ Theorem

Assuming F is a PRF, CTR mode is a CPA secure SKE for VIL.

📖 Proof

Let $G(\lambda, b) \equiv \text{GAME}_{\Pi, \mathcal{A}}^{\text{cpa}}(\lambda, b)$ where Π is CTR mode using F .

We need to show $G(\lambda, 0) \approx_c G(\lambda, 1)$.

Recall that in $G(\lambda, 0)$:

- Upon input an encryption query $m = (m_1, \dots, m_d)$, we return a ciphertext $c = (c_0, c_1, \dots, c_d)$ such that $c_0 = r \in U_n$, $c_i = F_k(r + i - 1) \oplus m_i$.
- For the challenge $m_b^* = (m_{b_1}^*, \dots, m_{b_{d^*}}^*)$ ($d^* \in \mathbb{N}$ is the dimension). Then $c^* = (c_0^*, c_1^*, \dots, c_{d^*}^*)$, with $c_0^* = r^* \in U_n$; $c_i^* = F_k(r^* + i - 1) \oplus m_{b,i}^*$

Let $H_1(\lambda, b)$ same as $G(\lambda, b)$ but replace $I_k(\cdot)$ with $R(\cdot)$ such that $R \leftarrow \mathcal{R}(\lambda, n \rightarrow ?)$

$H_2(\lambda)$: The challenge ctx c^* is uniform and independent of b :

🔗 Lemma

$\forall b \in \{0, 1\}$, $H_1(\lambda, b) \approx_c G(\lambda, b)$.

📖 Proof

Fix b . Assume (by contradiction) $\exists PPT$ such that it tells apart $H_1(\lambda, b)$ from $G(\lambda, b)$ with $Pr > \frac{1}{poly(\lambda)}$.

Then we build PPT $\mathcal{A}_{prf}$ against F . $\mathcal{A}_{prf}$ tells $\mathcal{A}$ to start its procedures, and sends the value $r \in U_n$ to the challenger $\mathcal{C}_{prf}$. $\mathcal{A}$ will send back $m = (m_1, \dots, m_d)$ and between $\mathcal{A}_{prf}$ and $\mathcal{C}_{prf}$ there will be an exchange of r (increased by 1 for each round), and values of z_i (so, for r $\mathcal{C}_{prf}$ will send z_1 , for $r+1$ we have z_2 , and so on). $\mathcal{A}_{prf}$ will compute $c_i = z_i \oplus m_i$ and will send $c = (r, c_1, \dots, c_d)$ to $\mathcal{A}$. $\mathcal{A}$ will answer with m_0^*, m_1^* which $\mathcal{A}_{prf}$ will answer with c^* , and finally $\mathcal{A}$ will output b' , which will be sent to the challenger by $\mathcal{A}_{prf}$.

🔗 Lemma

$$H_1(\lambda, b) \approx_c H_2(\lambda), \forall b \in \{0, 1\}$$

as long as # encryption queries = $2^\lambda = poly(\lambda)$

(isto é stat dist. q. comp. unbounded)
 - Kad os desafios iniciais de $G(\lambda, 0) \approx_c H_1(\lambda, 0)$
 $\approx_c H_2(\lambda) \approx_c H_1(\lambda, 1) \approx_c G(\lambda, 1)$

📖 Proof

Let's look at the sequence of values that are XORed to create the challenge ?

$H_1(x, b)$.

$R(r^*), R(r^* + 1), \dots, R(r^* + d^* - 1)$.

In contrast, during the encryption queries, we generate the sequence

$R(r_i), R(r_i + 1), \dots, R(r_i + d_i - 1)$

We define a bad event which becomes true if $\exists i, j, j' \geq 1$ such that $r_i + j = r^* + j'$ with

$$\begin{pmatrix} j = 0, \dots, d_i - 1 \\ j' = 0, \dots, d^* - 1 \end{pmatrix}$$

Now, conditioning on $\overline{BAD}$, then the sequence $r^*, r^* + 1, \dots, r^* + d^* - 1$ never overlaps, which means the challenge ciphertext is distributed like

$m_{b,1}^* \oplus u_1, \dots, m_{b,d^*}^* \oplus u_d$ which is identical to $U_{n,d}$ as in $H_2(\lambda)$.

A consequence of this is that $SD(H_1(\lambda, b); H_2(\lambda)) \leq Pr(BAD)$.

It remains to compute $Pr[BAD]$.

Fix i , and let BAD_i be the event that V^* sequence overlaps with the r_i sequence.

Wlog assume that $d_i = a^* = ? = poly(\lambda)$. The event BAD_i is the event that:

$r^*, r^* + 1, \dots, r^* + ? - 1$ overlaps $r_i, r_i + 1, \dots, r_i + ? - 1$.

$$\begin{aligned} &\implies r^* - ? + 1 \leq r_i \leq r^* + ? - 1 \\ &\implies Pr[BAD_i] \leq \frac{(r^* + ? - 1) - (r^* - ? + 1)}{2^n} = \frac{2? - 1}{2^n} \end{aligned}$$

Finally, by the union bound:

$$Pr[BAD] = Pr[\exists i \in [?] : BAD_i] \leq \sum_{i=1}^? Pr[BAD_i] \leq ? \cdot \frac{2? - 1}{2^n} \leq \frac{2^2}{2^n} = negl.$$

We now switch to the problem of message authentication.

$Tag : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{T}$

What's the security of Tag in the computational setting? We'll require that it is hard to forge a pair m^*, τ^* such that $Tag(k, m^*) = \tau^*$ without knowing k .

✓ Definition

We say $\Pi = Tag$ is UFCMA if

$$\forall PPT \mathcal{A}: Pr[GAME_{\Pi, \mathcal{A}}^{ufcma}(\lambda) = 1] \leq negl(\lambda)$$

So we have an adversary $\mathcal{A}$ who sends a message to a challenger $\mathcal{C}$ and he answers back by sending $\tau = Tag(k, m)$. This process takes place many times. Finally, $\mathcal{A}$ sends to $\mathcal{C}$ (m^*, τ^*) .

Output 1: $Tag(k, m^*) = \tau^*$ with $m' \neq m$ (already seen m).

✓ Theorem

If $F = \{F_k\}$ is a PRF, then $TAG(k, m) = F_k(m)$ is a UFCMA (for fixed input length (FIL))